

The Coding Toolbox.

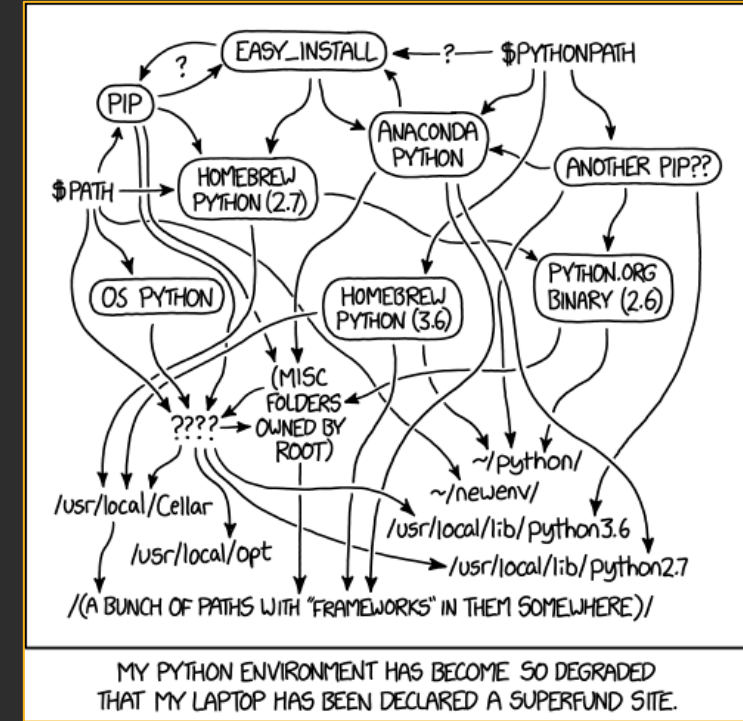
Week 1 | Lecture 2 (1.2)

Upcoming

- Lab 0 Released 6 pm Thursday.
- Reflection 1 Released Friday 6:00 pm.
- Tutorial sessions running all week.

if nothing else, write **#cleancode**.

- **Lecture 1.1**
 - Introduction to APS106
- **Lecture 1.2**
 - Computers, Programs, and The Coding Toolbox
- **Lecture 1.3**
 - Variables, Expressions, and Operators



What you will get out of today's lecture.

- Everything you need to begin coding in APS106.
- Learn about the core building blocks of a computer.
- Develop computational language and vocabulary for the files and programs used in **APS106**.
- Learn about the key differences in the software and tools used in APS106.
- Get experience opening and using the most important tools.
- Execute your first files of APS106!
- Create a folder structure for APS106.
- **Assuming You Have Installed Anaconda and VSCode.**
 - **If not, try to raise your hand for support.**

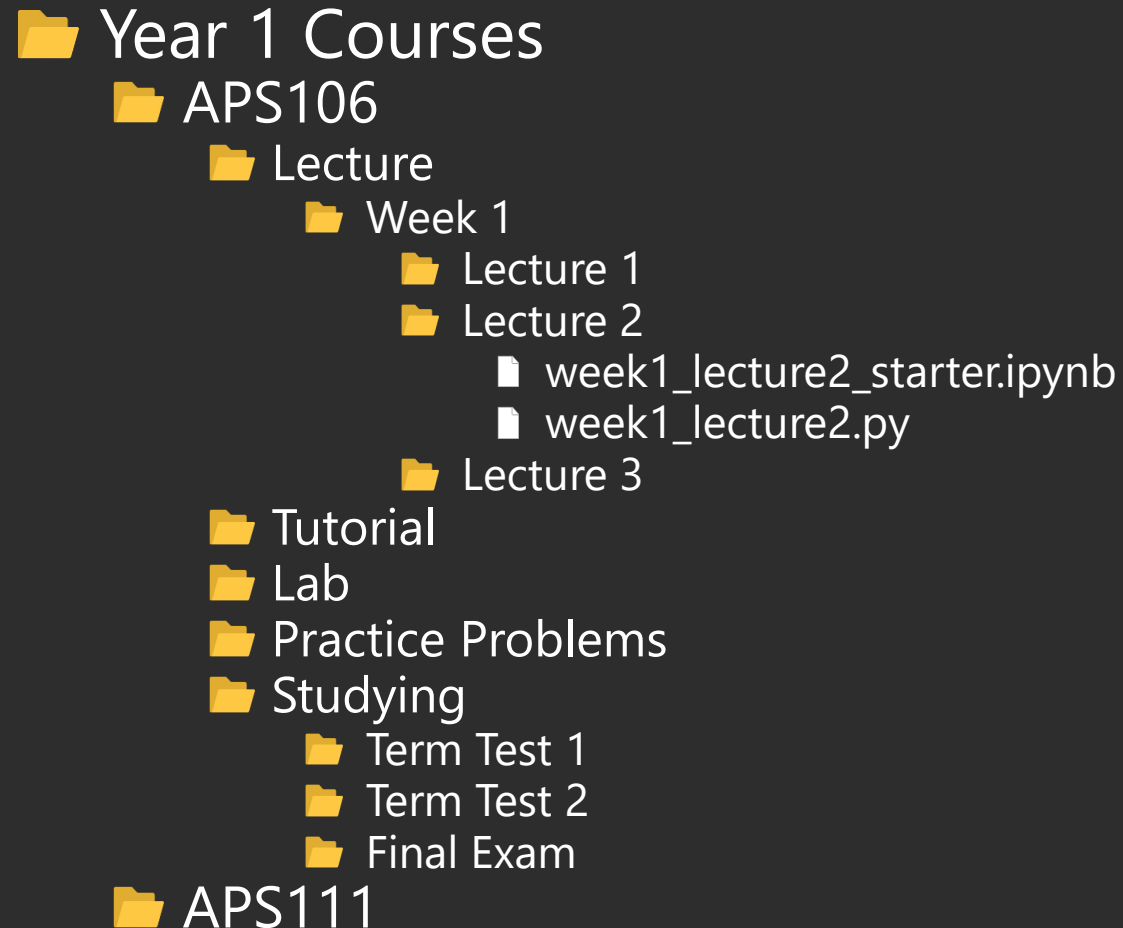
APS106 Folder Structure.

Downloads

- 34567543.jpg
- aaaaaaaaaaaaaaaaaa.py
- week1_lecture2.ipynb
- week1_lecture1.pdf
- week1_lecture2(1).ipynb
- week1_lecture3.ipynb
- please_work2.py
- taxes_2024_please_dont_audit_me.xlsx
- week12_lecture3(5).ipynb
- APS111_Slides.pptx
- group_project_final_by_me_because_no_one_else_did_it.py
- crypto_strategy_guaranteed_profit.docx
- screenshot_2026-01-04_1315.png
- python_stuff.zip
- lab2_fixed_FIXED2_actuallyfixed.py
- passwords.txt (**"Don't do this."**)

**DON'T
DO
THIS.**

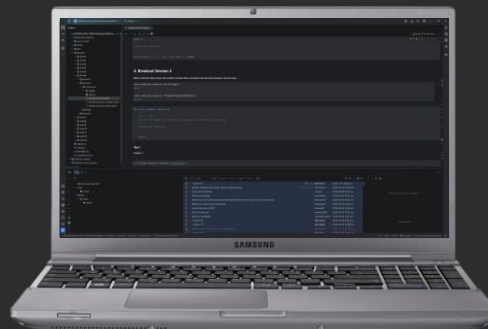
APS106 Folder Structure.



DO
THIS.
NOW.

Developer Toolkit.

Computer.



Version Control.



Git + GitHub

Editor/IDE.



VSCode + Jupyter

Developer Seb.



Python Interpreter.



Packages Manager.



ANACONDA®

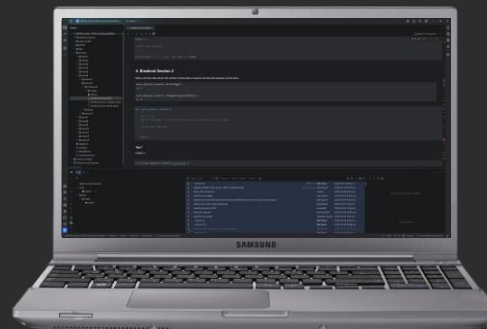
Terminal.



Anaconda Terminal

Developer Toolkit.

Computer.



Python Interpreter.



Version Control.



Git + GitHub

Packages Manager.



ANACONDA

Terminal.



Anaconda Terminal



Developer Seb.

Editor/IDE.



VSCode + Jupyter

Inside your computer?

■ Storage.

- Disks, hard drives, thumb drives, etc.
- Stores long-term data (files, programs, movies, etc.).
- Non-volatile: maintains contents when powered off.

■ Memory.

- Random Access Memory (RAM).
- Stores short-term data required for open programs and processes.
- Volatile: loses contents when powered off.

Software programs are usually stored and executed from RAM.

Later this term, we will learn how to read from and write to long-term storage.



Inside your computer?

World records for the number of transistors on a chip are held by U of T researchers in ECE!

- **Central Processing Unit (CPU).**
 - Runs the programs, including the operating system.
 - Reads and writes the data between storage and memory.
 - Composed of billions of miniature electronic switches called transistors.
 - Popular examples include Intel's Core i7 and AMD's Ryzen 7.
- **Clock.**
 - Controls the rate at which processors execute instructions (recall: programming is just a series of instructions).
 - Modern computers have clocks that beat around 4 GHz (4 billion times per second!).



Inside your computer?

■ Graphics Processing Unit (GPU).

- A processor originally designed to render images by doing the same math operation on many pixels at once.
- Training neural networks is dominated by matrix multiplications and vector operations (linear algebra), which map extremely well to GPU parallel hardware.
- A CPU has a small number of powerful cores optimized for general-purpose, branching tasks (running an OS, game logic, control flow), while a GPU has many more smaller cores optimized for massively parallel math (like the matrix operations that dominate deep learning).



3dfx Voodoo2.
(8 MB)



188.85 USD

+188.81 (472,025.00%) ↑ all time

Closed: Jan 2, 7:59 p.m. EST • [Disclaimer](#)

After hours 188.97 +0.12 (0.064%)

Nvidia RTX 2080 Ti.
(11 GB)



Outside your computer?

■ Input Devices.

- Keyboard.
- Mouse.
- Trackpad.
- Touch screen.
- Webcam.
- Gesture trackers.
- Eye trackers.



■ Output Devices.

- Monitors.
- Printers.
- Headphones.
- Speakers.
- Video/sound cards.
- Braille readers.



File Formats and Extensions.

- Files store data in different formats based on specific definitions.
- An "extension" is the file name's suffix, or the set of characters (usually 2-4) at the end of a file name, separated by a period (.).

File Types	Common Extension Examples
Plain text file	.txt
Document file	.doc, .docx, .pdf
Image file	.jpg, .gif, .png
Compressed/Archive file	.zip, .rar, .7z
Video file	.mpg, .mp4, .avi
Executable file	.exe, .bat, .bin
Python file	.py
Jupyter Notebook file	.ipynb
Comma Separated Value file	.csv

File extensions help indicate:

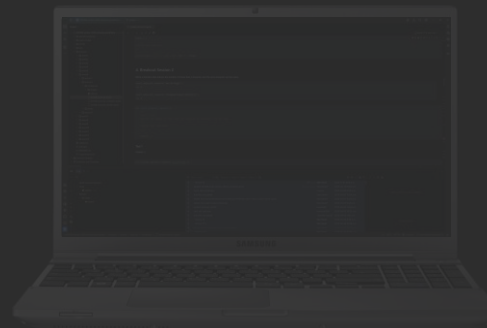
- The file format.
- How the data is structured.
- Which programs can open it?

Developer Toolkit.

Python Interpreter.



Computer.



Version Control.



Git + GitHub

Packages Manager.



ANACONDA

Terminal.



Anaconda Terminal



Developer Seb.

Editor/IDE.



VSCode + Jupyter

Python Interpreter.

- You write code in a **.py** file (plain text).
- In the terminal you run:
 - **python myfile.py**
- The Python interpreter reads your file top to bottom, translating it into actions as it goes.
- It executes the instructions (does calculations, calls functions, reads input, etc.).
- Results appear as output (printed text, files saved, graphs shown), and if something is wrong you get an error message/traceback.

test_file.py

```
print("Hello Class!")
```



standard output
(stdout)

Hello Class!

Python Interpreter.

- You write code in a `.py` file (plain text).
- In the terminal you run:
 - `python myfile.py`
- The Python interpreter reads your file top to bottom, translating it into actions as it goes.
- It executes the instructions (does calculations, calls functions, reads input, etc.).
- Results appear as output (printed text, files saved, graphs shown), and if something is wrong you get an error message/traceback.

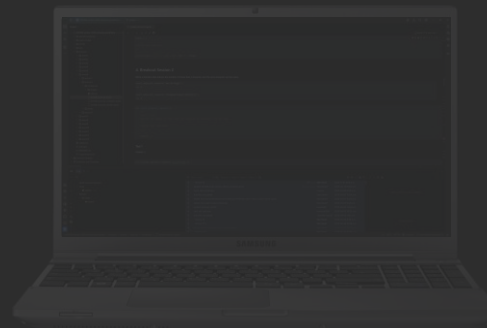
Let's Code!

Open:

1. **Anaconda Prompt**
2. **A Text Editor**

Developer Toolkit.

Computer.



Version Control.



Git + GitHub

Python Interpreter.



Packages Manager.



ANACONDA

Terminal.



Anaconda Terminal



Developer Seb.

Editor/IDE.



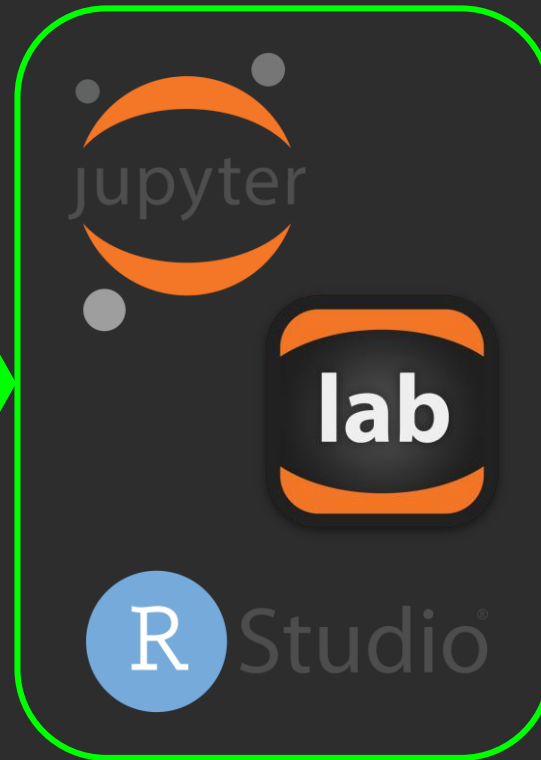
VSCode + Jupyter

Editors & IDEs.

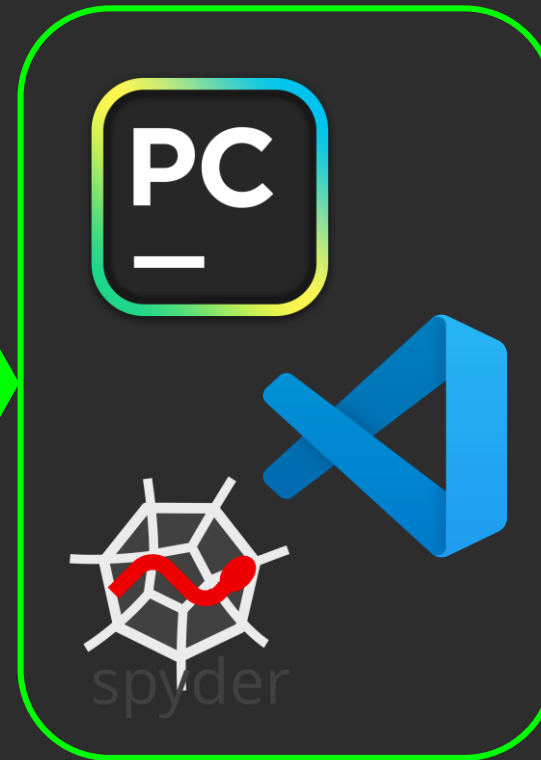
Text Editors.



Interactive Computing Environment.



Integrated Development Environment (IDE).



AI-First IDE.



Editors & IDEs.

Text Editors.



Interactive Computing Environment.



Integrated Development Environment (IDE).

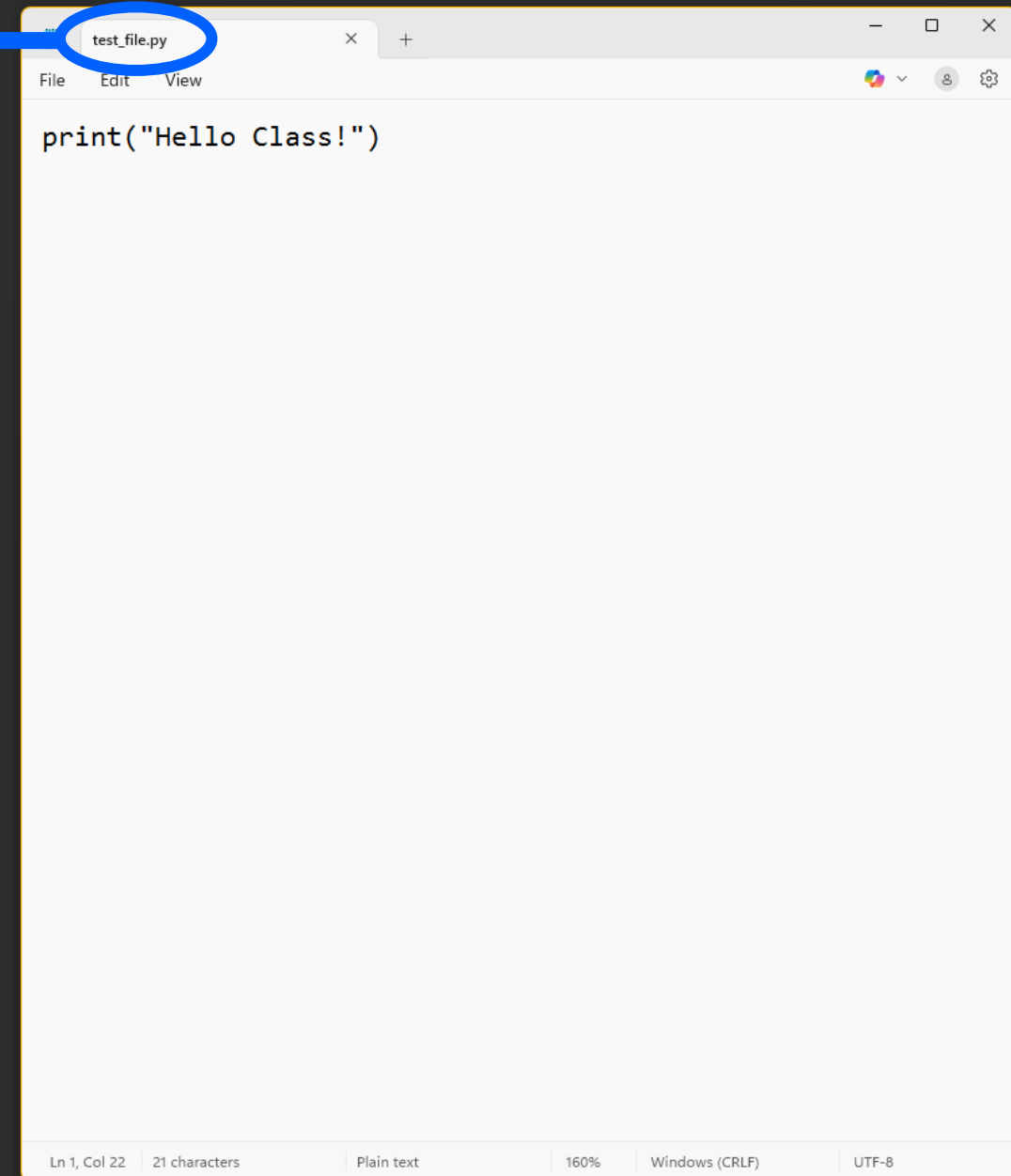


AI-First IDE.



Text Editor.

- **What it is:** A lightweight app for writing and editing plain-text files (including code), with minimal setup.
- **What you get:** Fast startup, syntax highlighting, find/replace, multiple files/tabs, basic project folders.
- **What you don't (usually) get:** Integrated debugging, deep refactoring, environment management, and one-click test runners.
- **Typical workflow:** Edit code in the editor, then run it in a separate terminal (`python myfile.py`).
- **Best for:** Quick edits, small scripts, configuration files, and learning fundamentals without heavy tooling.



Text Editor.

- **What it is:** A lightweight app for writing and editing plain-text files (including code), with minimal setup.
- **What you get:** Fast startup, syntax highlighting, find/replace, multiple files/tabs, basic project folders.
- **What you don't (usually) get:** Integrated debugging, deep refactoring, environment management, and one-click test runners.
- **Typical workflow:** Edit code in the editor, then run it in a separate terminal (`python myfile.py`).
- **Best for:** Quick edits, small scripts, configuration files, and learning fundamentals without heavy tooling.

Week 1 Lectures (LEC0101)

Slides and starter code notebooks will be posted before each lecture.

Lecture 1.1

[Slides](#) ↓

Lecture 1.2

[Slides](#) ↓

[Python File Demo \(.py\)](#) ↓

[Jupyter Notebook File Demo \(JupyterHub\)](#)

[Jupyter Notebook File Demo \(.ipynb\)](#) ↓

Lecture 1.3

[Slides](#) ↓

[Jupyter Notebook Starter \(JupyterHub\)](#)

[Jupyter Notebook Starter \(.ipynb\)](#) ↓

APS106

**File Type #1.
The Python File
.py**



Text Editor.

- **What it is:** A lightweight app for writing and editing plain-text files (including code), with minimal setup.
- **What you get:** Fast startup, syntax highlighting, find/replace, multiple files/tabs, basic project folders.
- **What you don't (usually) get:** Integrated debugging, deep refactoring, environment management, and one-click test runners.
- **Typical workflow:** Edit code in the editor, then run it in a separate terminal (`python myfile.py`).
- **Best for:** Quick edits, small scripts, configuration files, and learning fundamentals without heavy tooling.

Let's Code!

DO:

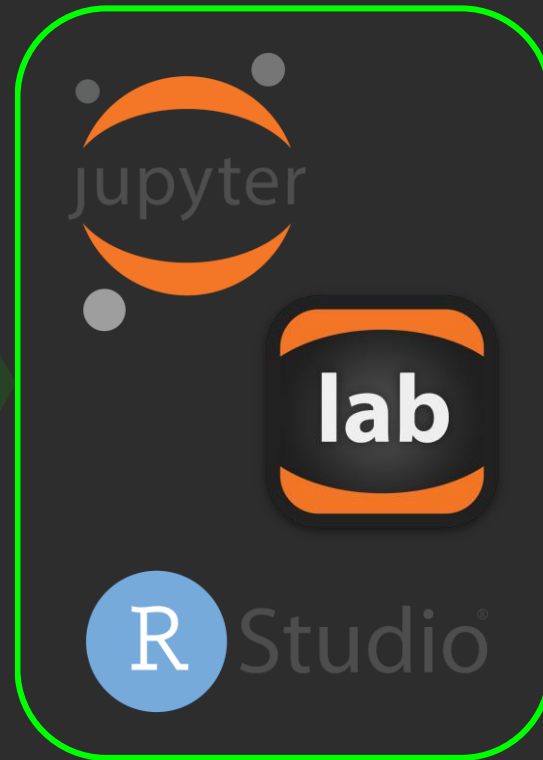
1. Download `week1_lecture2.py`
2. Open in a Text Editor
3. Open Anaconda Prompt
4. Run the file

Editors & IDEs.

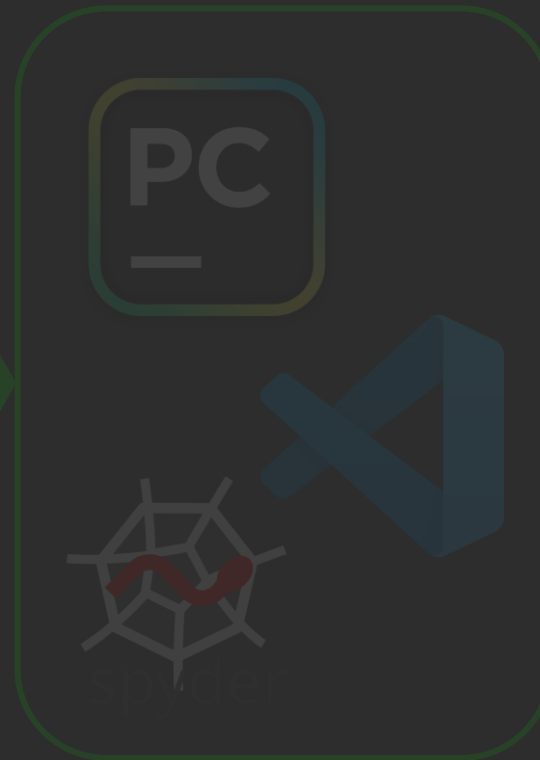
Text Editors.



Interactive
Computing
Environment.



Integrated
Development
Environment
(IDE).



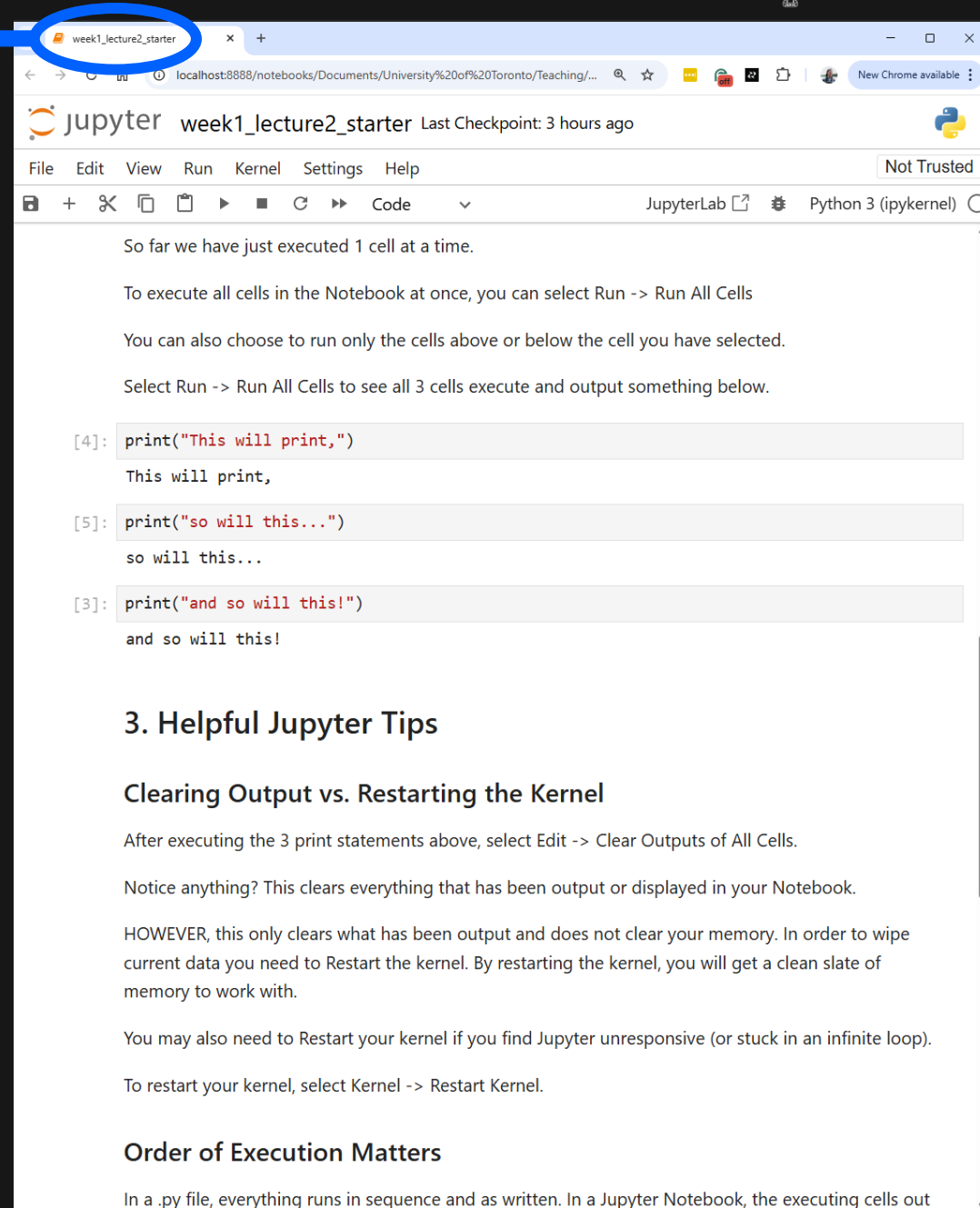
AI-First
IDE.



Interactive Computing Environment.

- **What it is:** An environment for running code interactively in small chunks (cells) while keeping notes and results in the same document.
- **How it works:** A notebook (.ipynb) connects to a live kernel (a running Python process) that remembers variables between cells.
- **What you get:** Immediate output (text/tables/plots), easy experimentation, and a natural format for explanation + code + results.
- **Typical workflow:** Write a cell → run it → inspect output → iterate; often used for data analysis, visualization, and demos.
- **Best for:** Learning, prototyping, data science/ML workflows, and reproducible “computational narratives”.
- **Key caution:** Execution order matters, running cells out of order can create confusing hidden state; “Restart Kernel + Run All” helps verify correctness.

File Type #2.
The Python File
.ipynb



week1_lecture2_starter

localhost:8888/notebooks/Documents/University%20of%20Toronto/Teaching/...

Jupyter week1_lecture2_starter Last Checkpoint: 3 hours ago

File Edit View Run Kernel Settings Help Not Trusted

Code Python 3 (ipykernel)

So far we have just executed 1 cell at a time.

To execute all cells in the Notebook at once, you can select Run -> Run All Cells

You can also choose to run only the cells above or below the cell you have selected.

Select Run -> Run All Cells to see all 3 cells execute and output something below.

```
[4]: print("This will print,")
This will print,
```

```
[5]: print("so will this...")
so will this...
```

```
[3]: print("and so will this!")
and so will this!
```

3. Helpful Jupyter Tips

Clearing Output vs. Restarting the Kernel

After executing the 3 print statements above, select Edit -> Clear Outputs of All Cells.

Notice anything? This clears everything that has been output or displayed in your Notebook.

HOWEVER, this only clears what has been output and does not clear your memory. In order to wipe current data you need to Restart the kernel. By restarting the kernel, you will get a clean slate of memory to work with.

You may also need to Restart your kernel if you find Jupyter unresponsive (or stuck in an infinite loop).

To restart your kernel, select Kernel -> Restart Kernel.

Order of Execution Matters

In a .py file, everything runs in sequence and as written. In a Jupyter Notebook, the executing cells out

Interactive Computing Environment.

- **What it is:** An environment for running code interactively in small chunks (cells) while keeping notes and results in the same document.
- **How it works:** A notebook (.ipynb) connects to a live kernel (a running Python process) that remembers variables between cells.
- **What you get:** Immediate output (text/tables/plots), easy experimentation, and a natural format for explanation + code + results.
- **Typical workflow:** Write a cell → run it → inspect output → iterate; often used for data analysis, visualization, and demos.
- **Best for:** Learning, prototyping, data science/ML workflows, and reproducible “computational narratives”.
- **Key caution:** Execution order matters, running cells out of order can create confusing hidden state; “Restart Kernel + Run All” helps verify correctness.

Week 1 Lectures (LEC0101)

Slides and starter code notebooks will be posted before each lecture.

Lecture 1.1

[Slides](#) ↓

Lecture 1.2

[Slides](#) ↓

[Python File Demo \(.py\)](#) ↓

[Jupyter Notebook File Demo \(JupyterHub\)](#)

[Jupyter Notebook File Demo \(.ipynb\)](#) ↓

APS106
File Type #2.
The Python
Notebook File
.ipynb

Lecture 1.3

[Slides](#) ↓

[Jupyter Notebook Starter \(JupyterHub\)](#)

[Jupyter Notebook Starter \(.ipynb\)](#) ↓

Interactive Computing Environment.

■ Three Kinds Of Cells.

■ Markdown.

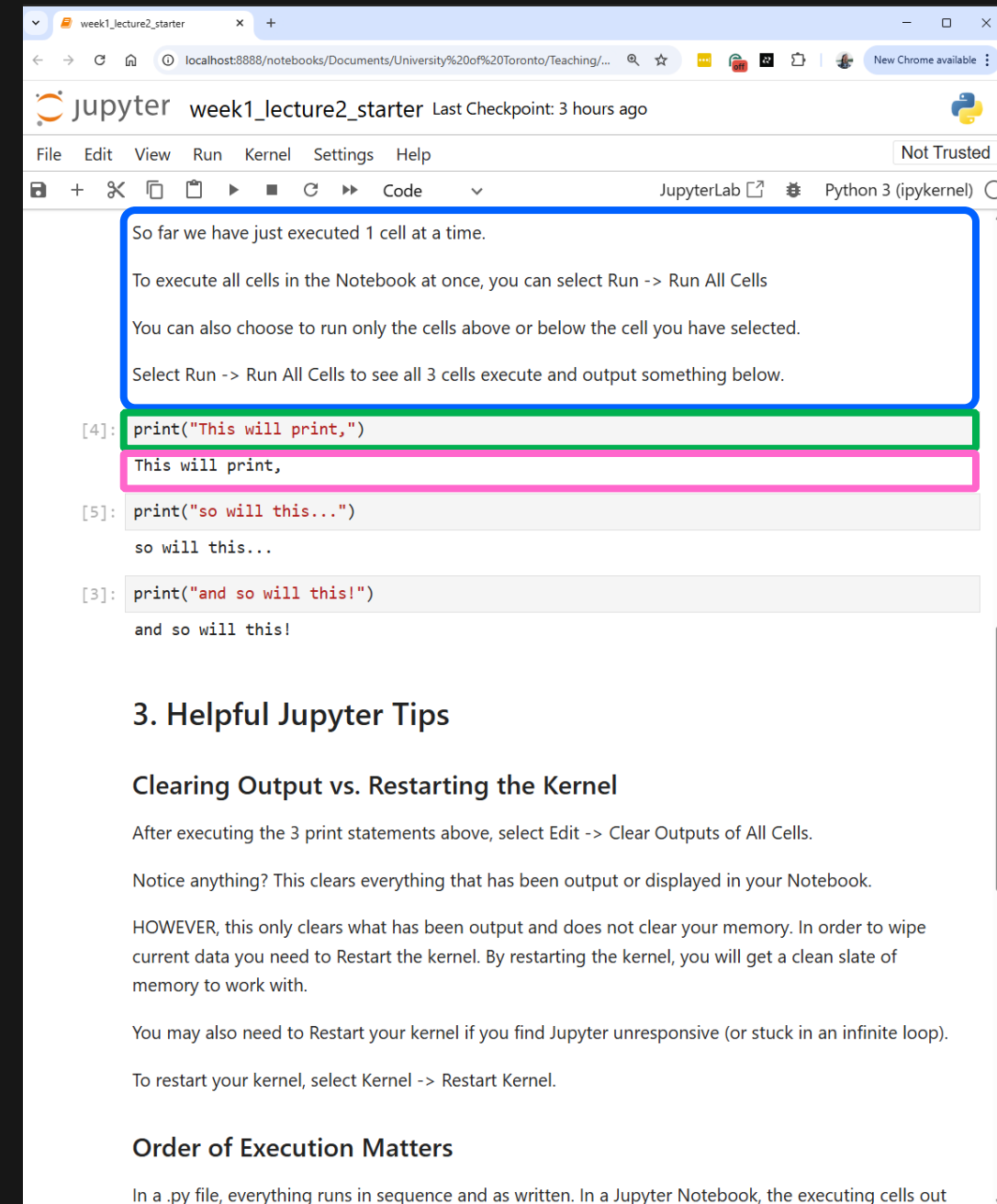
- Write formatted notes: headings, bullet lists, links, equations, and explanations (no code execution).

■ Code.

- Write and run Python (or other languages) in a cell; the kernel executes it and keeps variables in memory.

■ Output.

- The results produced by running a code cell, printed text, tables, plots, or error messages, are shown directly under the cell.



week1_lecture2_starter

localhost:8888/notebooks/Documents/University%20of%20Toronto/Teaching/...

jupyter week1_lecture2_starter Last Checkpoint: 3 hours ago

File Edit View Run Kernel Settings Help Not Trusted

JupyterLab Python 3 (ipykernel)

So far we have just executed 1 cell at a time.

To execute all cells in the Notebook at once, you can select Run -> Run All Cells

You can also choose to run only the cells above or below the cell you have selected.

Select Run -> Run All Cells to see all 3 cells execute and output something below.

```
[4]: print("This will print,")
This will print,
```

```
[5]: print("so will this...")
so will this...
```

```
[3]: print("and so will this!")
and so will this!
```

3. Helpful Jupyter Tips

Clearing Output vs. Restarting the Kernel

After executing the 3 print statements above, select Edit -> Clear Outputs of All Cells.

Notice anything? This clears everything that has been output or displayed in your Notebook.

HOWEVER, this only clears what has been output and does not clear your memory. In order to wipe current data you need to Restart the kernel. By restarting the kernel, you will get a clean slate of memory to work with.

You may also need to Restart your kernel if you find Jupyter unresponsive (or stuck in an infinite loop).

To restart your kernel, select Kernel -> Restart Kernel.

Order of Execution Matters

In a .py file, everything runs in sequence and as written. In a Jupyter Notebook, the executing cells out

Interactive Computing Environment.

■ Three Kinds Of Cells.

■ **Markdown.**

- Write formatted notes: headings, bullet lists, links, equations, and explanations (no code execution).

■ **Code.**

- Write and run Python (or other languages) in a cell; the kernel executes it and keeps variables in memory.

■ **Output.**

- The results produced by running a code cell, printed text, tables, plots, or error messages, are shown directly under the cell.

Let's Code!

Download:

1. [week1_lecture2_starter.ipynb](#)

Open:

1. **Jupyter App**

JupyterHub.

- **Jupyter Notebook:** The notebook application itself, an interface to create and run `.ipynb` files on a machine (your laptop or a server), typically for a single user.
- **JupyterHub:** A multi-user platform that hosts Jupyter Notebook/Lab for a class or organization, handles logins, starts a separate notebook server for each user, and runs everything on shared infrastructure (Cloud).

Week 1 Lectures (LEC0101)

Slides and starter code notebooks will be posted before each lecture.

Lecture 1.1

[Slides](#) ↓

Lecture 1.2

[Slides](#) ↓

[Python File Demo \(.py\)](#) ↓

[Jupyter Notebook File Demo \(JupyterHub\)](#) ←

[Jupyter Notebook File Demo \(.ipynb\)](#) ↓

**What is this
JupyterHub
thing?**

Lecture 1.3

[Slides](#) ↓

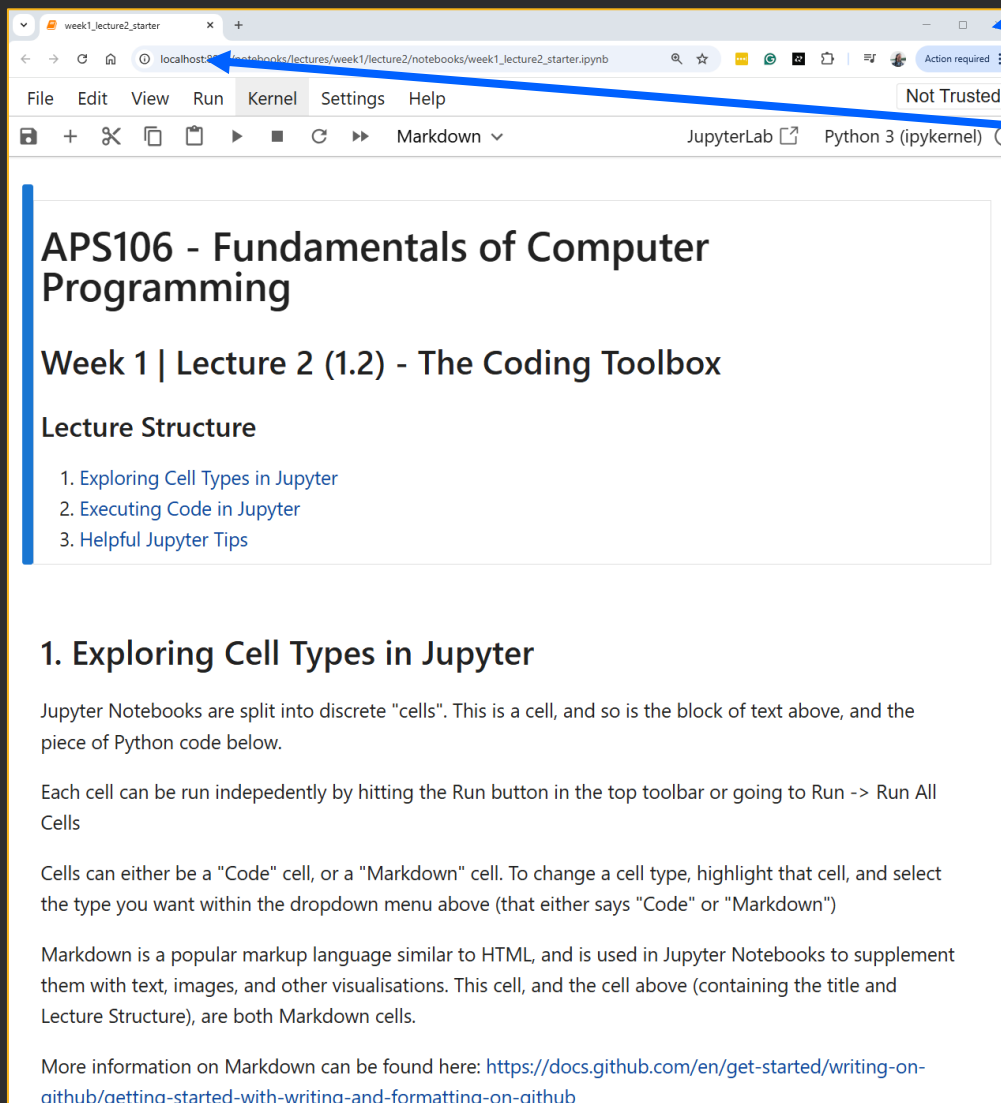
[Jupyter Notebook Starter \(JupyterHub\)](#)

[Jupyter Notebook Starter \(.ipynb\)](#) ↓

Jupyter Notebook.

Running in
the browser.

JupyterHub.



The screenshot shows a web browser window with the URL `localhost:8888/notebooks/lectures/week1/lecture2/notebooks/week1_lecture2_starter.ipynb`. The notebook interface includes a top toolbar with 'File', 'Edit', 'View', 'Run', 'Kernel', 'Settings', and 'Help' menus. Below the toolbar, the notebook content is displayed, starting with the title 'APS106 - Fundamentals of Computer Programming' and 'Week 1 | Lecture 2 (1.2) - The Coding Toolbox'. The 'Lecture Structure' section lists three items: '1. Exploring Cell Types in Jupyter', '2. Executing Code in Jupyter', and '3. Helpful Jupyter Tips'. The first section, '1. Exploring Cell Types in Jupyter', is expanded, showing a paragraph about Jupyter notebooks being split into discrete 'cells' and a list of instructions for running cells and changing cell types. The bottom of the notebook contains a link to GitHub documentation on Markdown.

APS106 - Fundamentals of Computer Programming

Week 1 | Lecture 2 (1.2) - The Coding Toolbox

Lecture Structure

1. Exploring Cell Types in Jupyter
2. Executing Code in Jupyter
3. Helpful Jupyter Tips

1. Exploring Cell Types in Jupyter

Jupyter Notebooks are split into discrete "cells". This is a cell, and so is the block of text above, and the piece of Python code below.

Each cell can be run independently by hitting the Run button in the top toolbar or going to Run -> Run All Cells

Cells can either be a "Code" cell, or a "Markdown" cell. To change a cell type, highlight that cell, and select the type you want within the dropdown menu above (that either says "Code" or "Markdown")

Markdown is a popular markup language similar to HTML, and is used in Jupyter Notebooks to supplement them with text, images, and other visualisations. This cell, and the cell above (containing the title and Lecture Structure), are both Markdown cells.

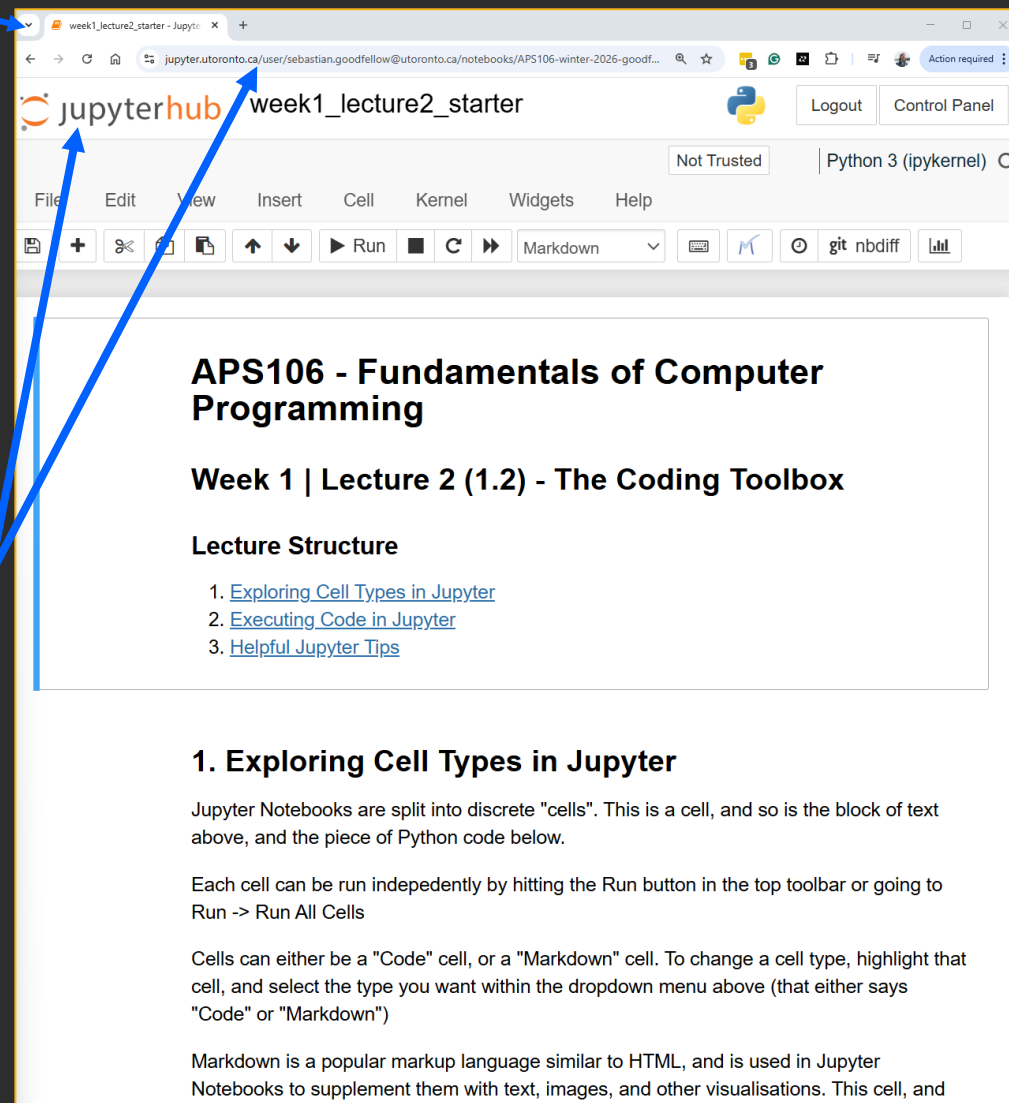
More information on Markdown can be found here: <https://docs.github.com/en/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github>

localhost means the computation is happening on your computer.

http://localhost:8888/notebooks/lectures/week1/lecture2/notebooks/week1_lecture2_starter.ipynb

jupyter.utoronto.ca means the computation is happening on another server.

https://jupyter.utoronto.ca/user/sebastian.goodfellow@utoronto.ca/notebooks/APS106-winter-2026-goodfellow-sutton/lectures/week1/lecture2/notebooks/week1_lecture2_starter.ipynb



The screenshot shows a web browser window with the URL `jupyter.utoronto.ca/user/sebastian.goodfellow@utoronto.ca/notebooks/APS106-winter-2026-goodfellow-sutton/lectures/week1/lecture2/notebooks/week1_lecture2_starter.ipynb`. The notebook interface is similar to the localhost version, but the top toolbar includes additional options like 'Insert', 'Cell', 'Kernel', 'Widgets', and 'Help'. The content is identical to the localhost version, starting with the title 'APS106 - Fundamentals of Computer Programming' and 'Week 1 | Lecture 2 (1.2) - The Coding Toolbox'. The 'Lecture Structure' section lists three items: '1. Exploring Cell Types in Jupyter', '2. Executing Code in Jupyter', and '3. Helpful Jupyter Tips'. The first section, '1. Exploring Cell Types in Jupyter', is expanded, showing a paragraph about Jupyter notebooks being split into discrete 'cells' and a list of instructions for running cells and changing cell types. The bottom of the notebook contains a link to GitHub documentation on Markdown.

APS106 - Fundamentals of Computer Programming

Week 1 | Lecture 2 (1.2) - The Coding Toolbox

Lecture Structure

1. Exploring Cell Types in Jupyter
2. Executing Code in Jupyter
3. Helpful Jupyter Tips

1. Exploring Cell Types in Jupyter

Jupyter Notebooks are split into discrete "cells". This is a cell, and so is the block of text above, and the piece of Python code below.

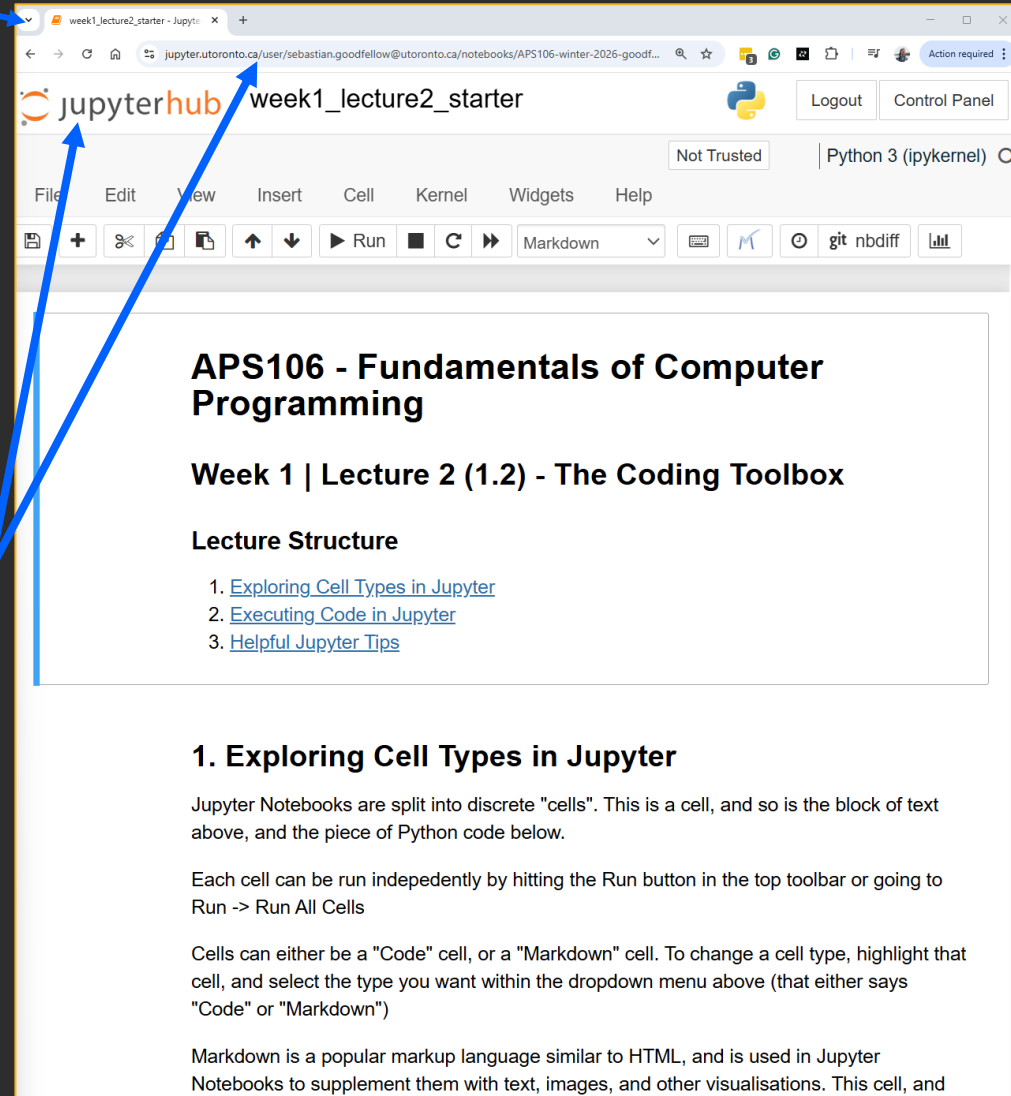
Each cell can be run independently by hitting the Run button in the top toolbar or going to Run -> Run All Cells

Cells can either be a "Code" cell, or a "Markdown" cell. To change a cell type, highlight that cell, and select the type you want within the dropdown menu above (that either says "Code" or "Markdown")

Markdown is a popular markup language similar to HTML, and is used in Jupyter Notebooks to supplement them with text, images, and other visualisations. This cell, and

JupyterHub.

Running in
the browser.



The screenshot shows a web browser window with the URL `jupyter.utoronto.ca/user/sebastian.goodfellow@utoronto.ca/notebooks/APS106-winter-2026-goodfellow-sutton/lectures/week1/lecture2/notebooks/week1_lecture2_starter.ipynb`. The page title is "week1_lecture2_starter". The interface includes a top bar with "jupyterhub" logo, "Logout", and "Control Panel" buttons. Below the top bar is a menu bar with "File", "Edit", "View", "Insert", "Cell", "Kernel", "Widgets", and "Help". A toolbar contains icons for file operations, a "Run" button, and a "Markdown" dropdown. The main content area displays the notebook title "APS106 - Fundamentals of Computer Programming", the section "Week 1 | Lecture 2 (1.2) - The Coding Toolbox", and a "Lecture Structure" list with three items: "1. Exploring Cell Types in Jupyter", "2. Executing Code in Jupyter", and "3. Helpful Jupyter Tips". Below this is a section titled "1. Exploring Cell Types in Jupyter" with text explaining Jupyter Notebooks and cell types.

Let's Code!

Demo:

1. JupyterHub

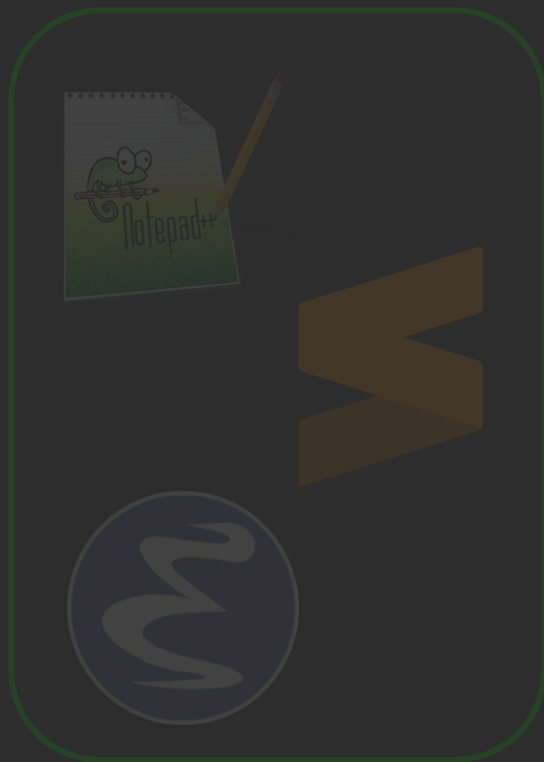
jupyter.utoronto.ca

means the
computation is
happening on
another server.

https://jupyter.utoronto.ca/user/sebastian.goodfellow@utoronto.ca/notebooks/APS106-winter-2026-goodfellow-sutton/lectures/week1/lecture2/notebooks/week1_lecture2_starter.ipynb

Editors & IDEs.

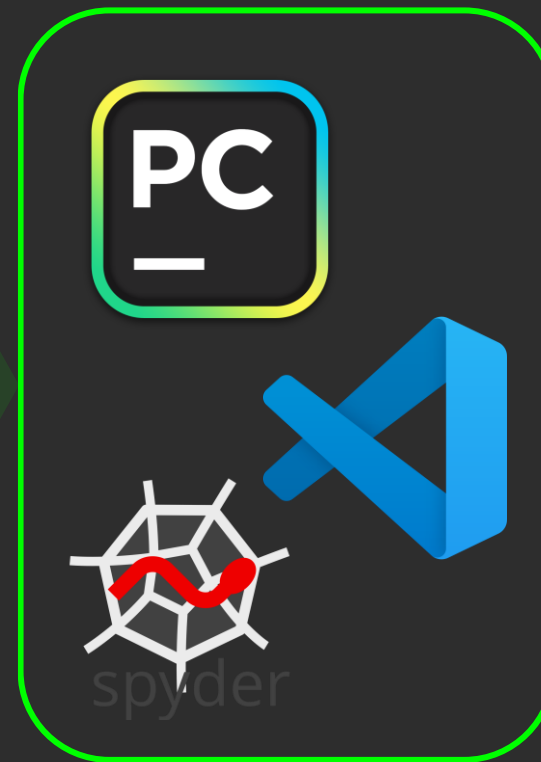
Text Editors.



Interactive
Computing
Environment.



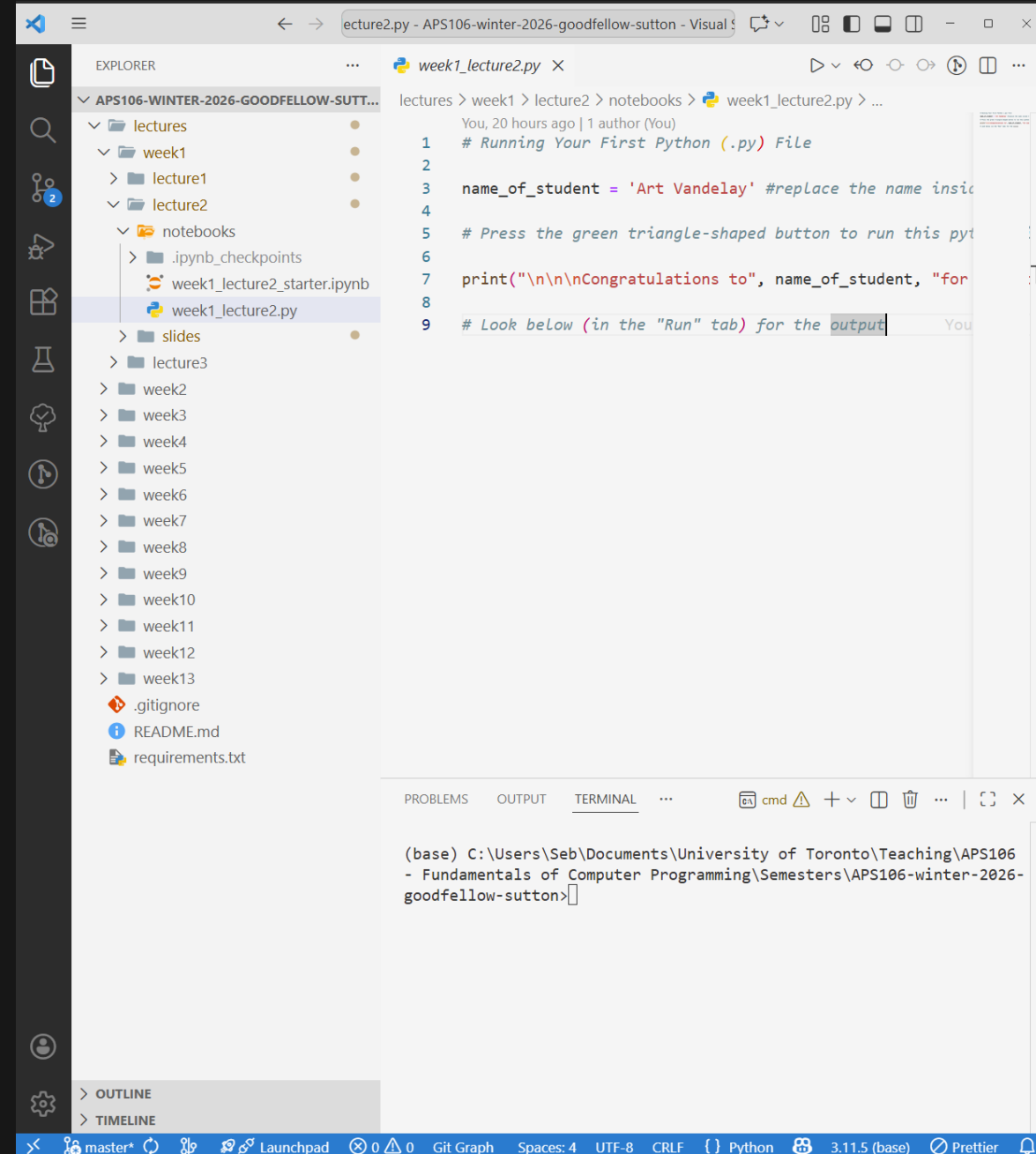
Integrated
Development
Environment
(IDE).



AI-First
IDE.



- **What it is:** A full-featured environment for building software, editing code, running it, debugging it, and managing a project in one place.
- **How it works:** An IDE understands your project (files, imports, dependencies) and integrates key tools: a run configuration, debugger, terminal, and often Git, testing, and environment selection.
- **What you get:** Autocomplete and code navigation, refactoring, integrated debugging (breakpoints/step-through), linting/formatting, and test runners, designed for scaling beyond a single file.
- **Typical workflow:** Open a project → edit code → run/debug → inspect variables and stack traces → write tests → commit changes to Git.
- **Best for:** Multi-file programs, assignments that grow over time, team projects, and any work where debugging and organization matter.



Integrated Development Environment.

- **What it is:** A full-featured environment for building software, editing code, running it, debugging it, and managing a project in one place.
- **How it works:** An IDE understands your project (files, imports, dependencies) and integrates key tools: a run configuration, debugger, terminal, and often Git, testing, and environment selection.
- **What you get:** Autocomplete and code navigation, refactoring, integrated debugging (breakpoints/step-through), linting/formatting, and test runners, designed for scaling beyond a single file.
- **Typical workflow:** Open a project → edit code → run/debug → inspect variables and stack traces → write tests → commit changes to Git.
- **Best for:** Multi-file programs, assignments that grow over time, team projects, and any work where debugging and organization matter.

Let's Code!

Open:

1. **VSCode**

Run:

1. **week1_lecture2.py**
2. **week1_lecture2_starter.ipynb**

Editors & IDEs.

Text Editors.



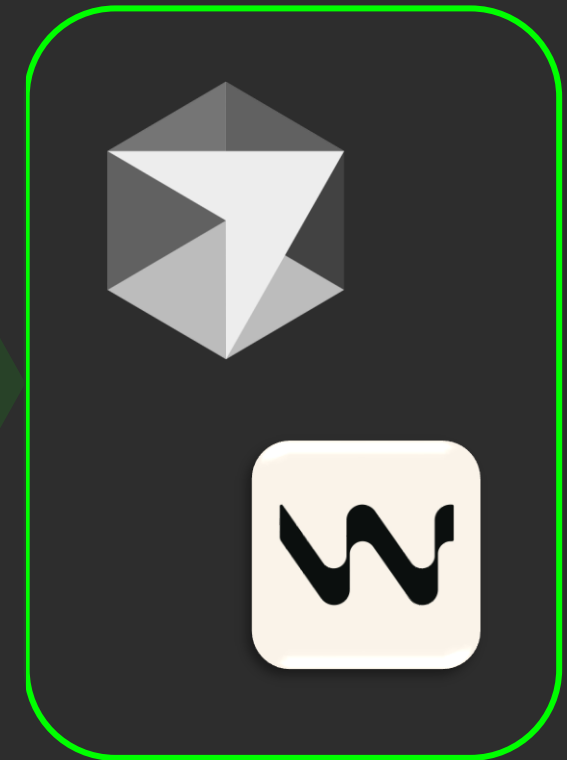
Interactive
Computing
Environment.



Integrated
Development
Environment
(IDE).

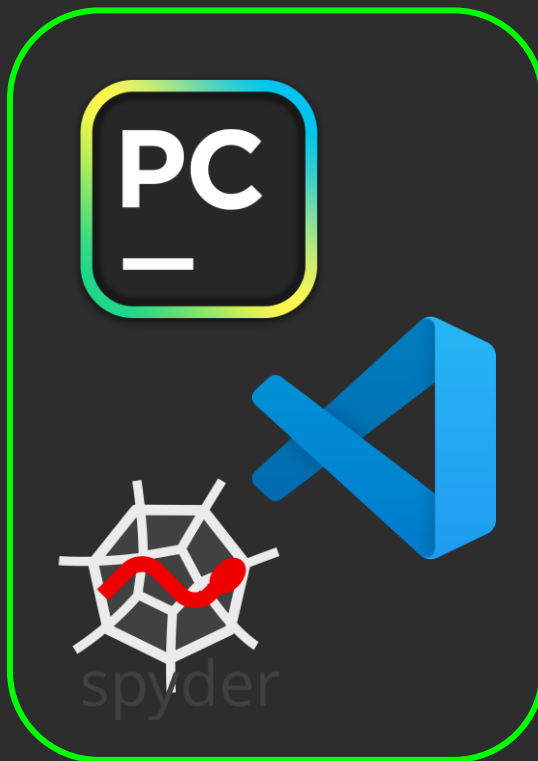


AI-First
IDE.



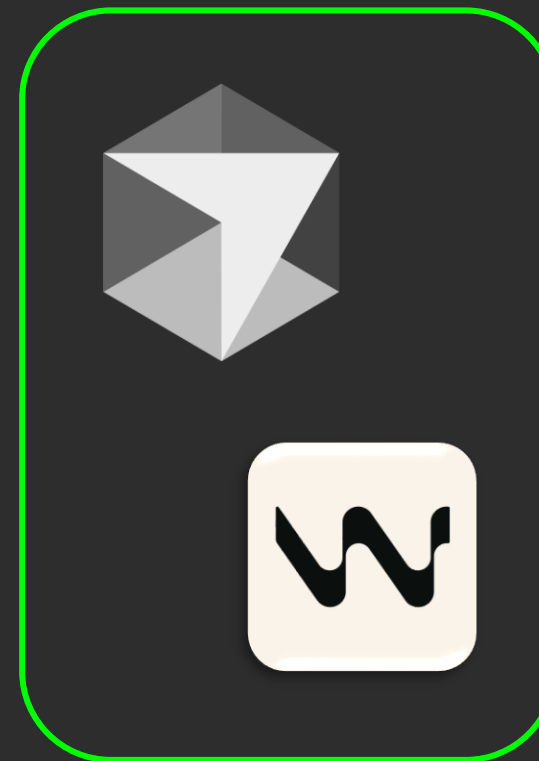
Editors & IDEs.

Integrated Development Environment (IDE).



These IDEs can become AI-enabled with extensions & CLI tools.

AI-First IDE.

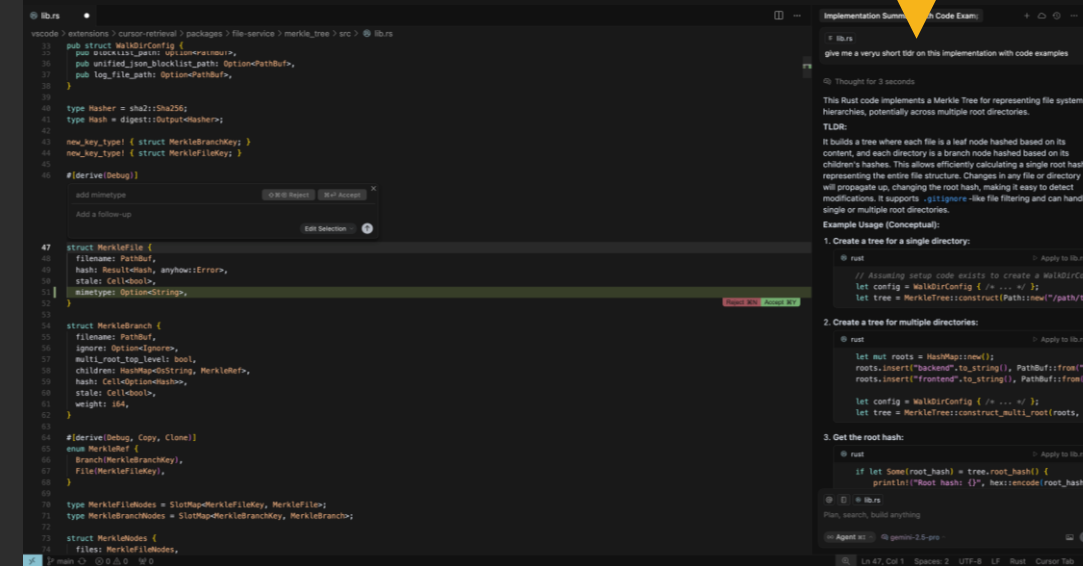


These IDEs have a UX that was built from the ground up for AI coding.

AI-First IDE.

- **What it is:** An IDE designed around an integrated AI “pair programmer” that can understand your codebase, suggest changes, and often make multi-file edits on your behalf.
- **How it works:** You prompt the AI in-chat (and/or highlight code); it uses codebase context to propose edits, generate new files, refactor across the project, and help run/debug, typically via an agent-style workflow.
- **What you get:** Faster scaffolding and iteration (boilerplate, examples, explanations), assisted debugging, quick refactors, and “search + understand” across a codebase with natural language.
- **Typical workflow:** Describe the change → AI proposes a diff across files → you review/accept edits → run tests → iterate with follow-up prompts.
- **Best for:** Rapid prototyping, learning patterns by example, refactoring, and reducing time spent on repetitive setup so you can focus on problem-solving.
- **Key caution:** The AI can be confidently wrong or introduce subtle bugs/security issues, treat outputs as drafts, review diffs carefully, and validate by running/tests (you still own correctness).

Embedded AI Agent.



AI-First IDE.

- **What it is:** An IDE designed around an integrated AI “pair programmer” that can understand your codebase, suggest changes, and often make multi-file edits on your behalf.
- **How it works:** You prompt the AI in-chat (and/or highlight code); it uses codebase context to propose edits, generate new files, refactor across the project, and help run/debug, typically via an agent-style workflow.
- **What you get:** Faster scaffolding and iteration (boilerplate, examples, explanations), assisted debugging, quick refactors, and “search + understand” across a codebase with natural language.
- **Typical workflow:** Describe the change → AI proposes a diff across files → you review/accept edits → run tests → iterate with follow-up prompts.
- **Best for:** Rapid prototyping, learning patterns by example, refactoring, and reducing time spent on repetitive setup so you can focus on problem-solving.
- **Key caution:** The AI can be confidently wrong or introduce subtle bugs/security issues, treat outputs as drafts, review diffs carefully, and validate by running/tests (you still own correctness).

Let's Code!

Demo:

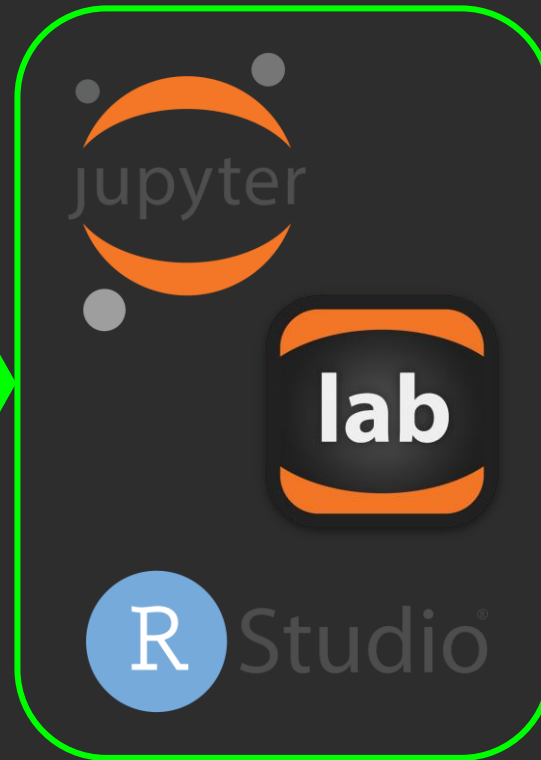
1. Cursor

Editors & IDEs.

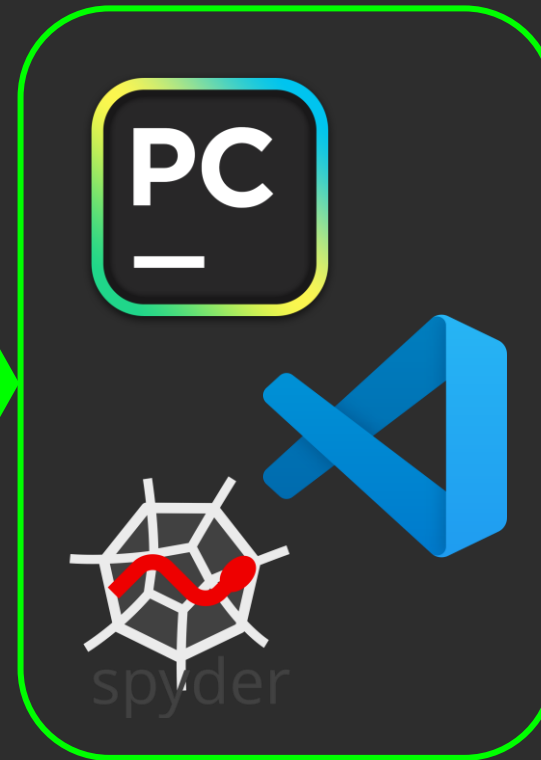
Text Editors.



Interactive Computing Environment.



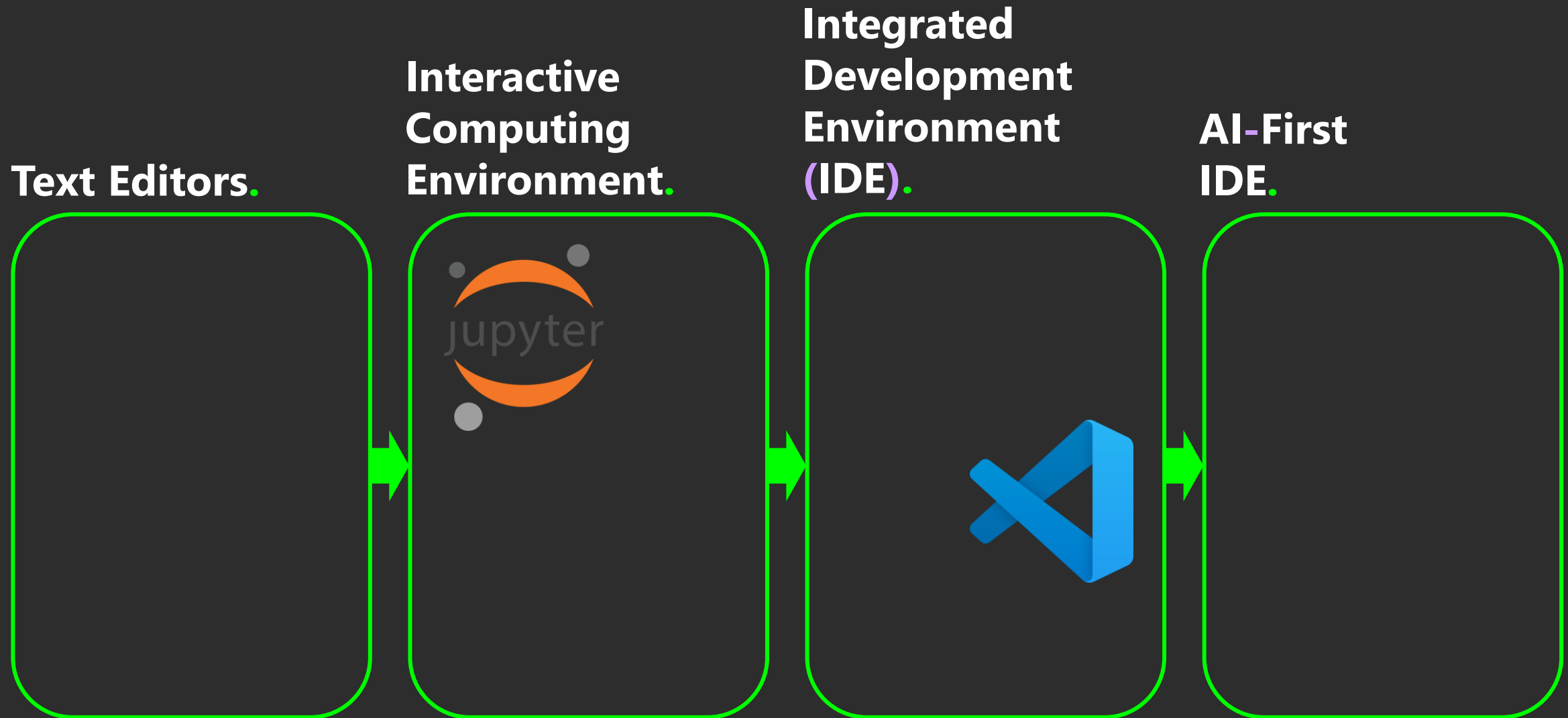
Integrated Development Environment (IDE).



AI-First IDE.

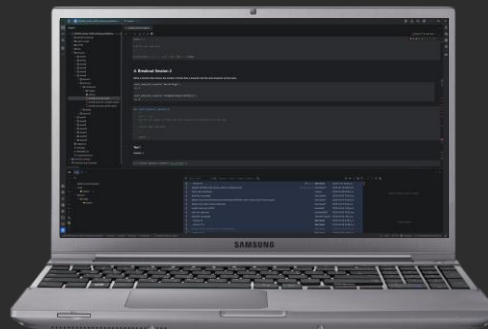


What We'll Use.



Developer Toolkit.

Computer.



Version Control.



Git + GitHub

Editor/IDE.



VSCode + Jupyter

Developer Seb.



Python Interpreter.



Packages Manager.



ANACONDA

Terminal.



Anaconda Terminal

Developer Toolkit.

Python Interpreter.



Packages Manager.



ANACONDA®

Terminal.



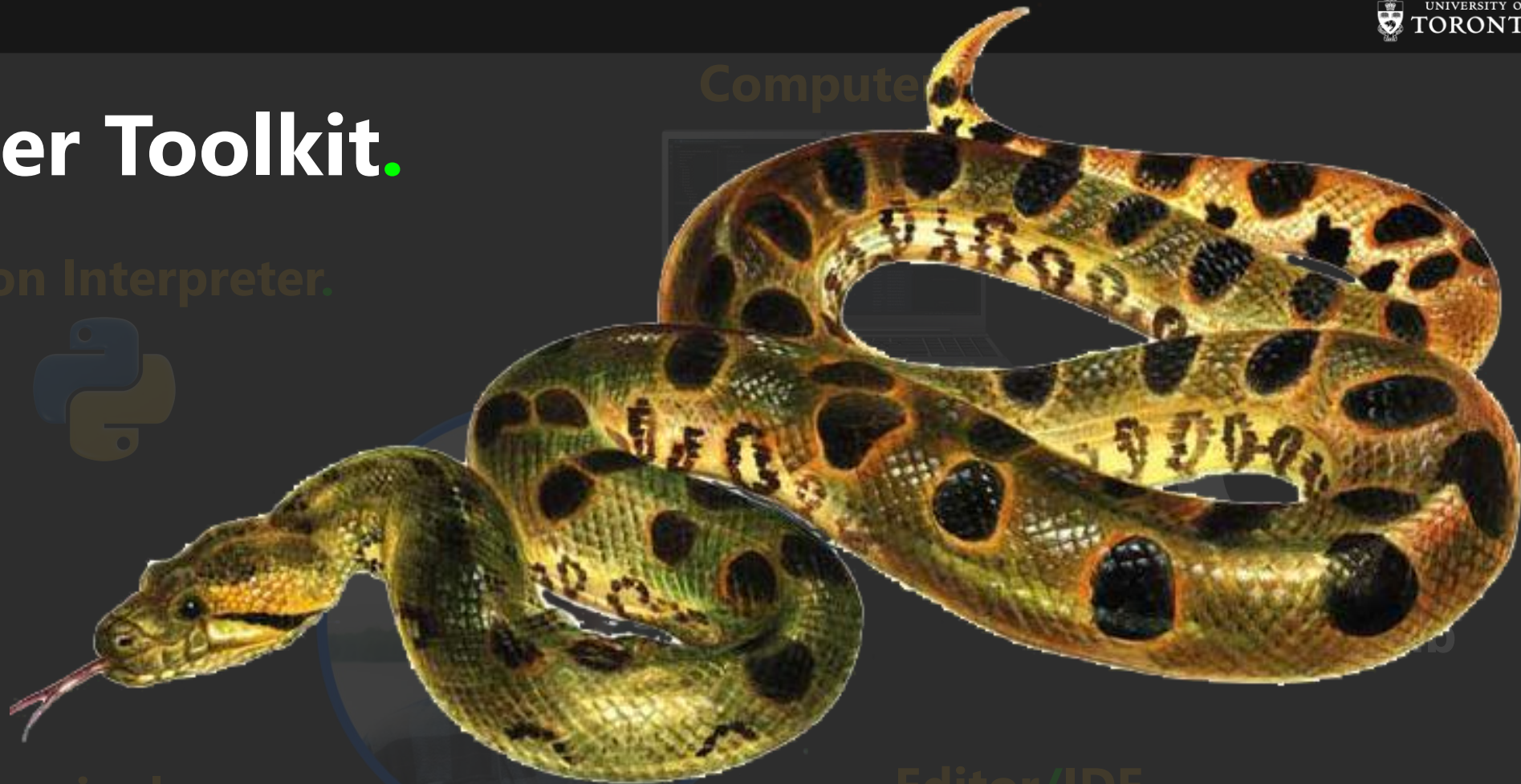
Anaconda Terminal

Developer Seb.

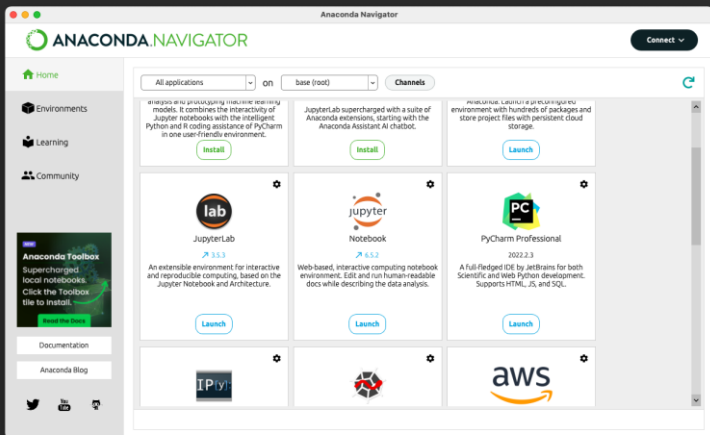
Editor/IDE.



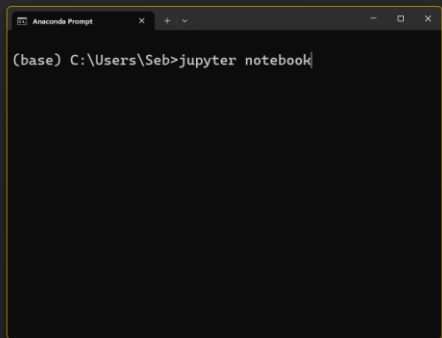
VSCode + Jupyter



Anaconda Navigator.



Anaconda Prompt.



Python Interpreter.



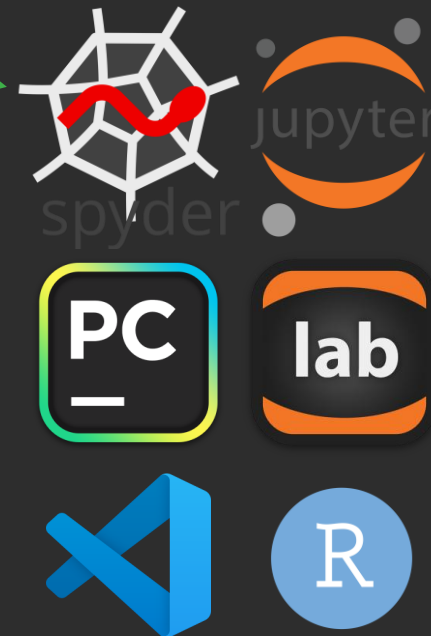
Package + Environment Manager.



Curated Package List. (base)



Editors.



ANACONDA

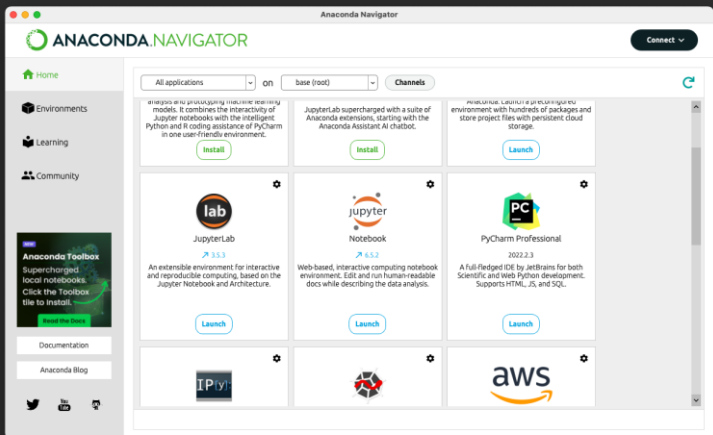
Anaconda is an all-in-one way to install and manage Python on your computer. It helps you set up the tools and libraries you need for coding without having to configure everything from scratch.



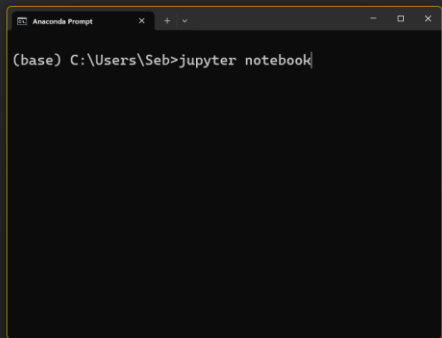
We could install
everything on
our own.



Anaconda Navigator.



Anaconda Prompt.



Python Interpreter.



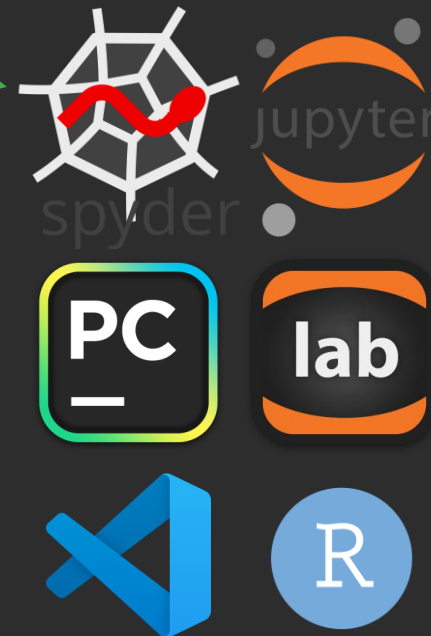
Package + Environment Manager.



Curated Package List. (base)



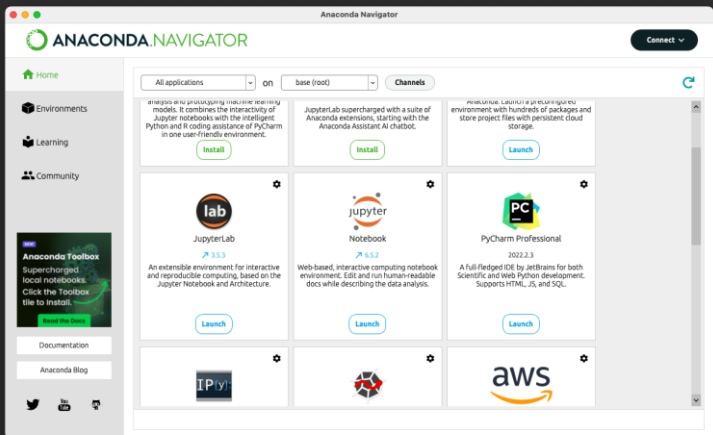
Editors.



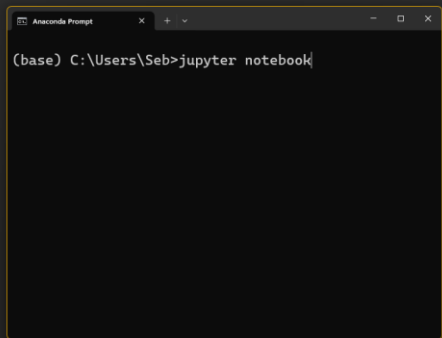
ANACONDA

But this is much easier!

Anaconda Navigator.



Anaconda Prompt.



Python Interpreter.



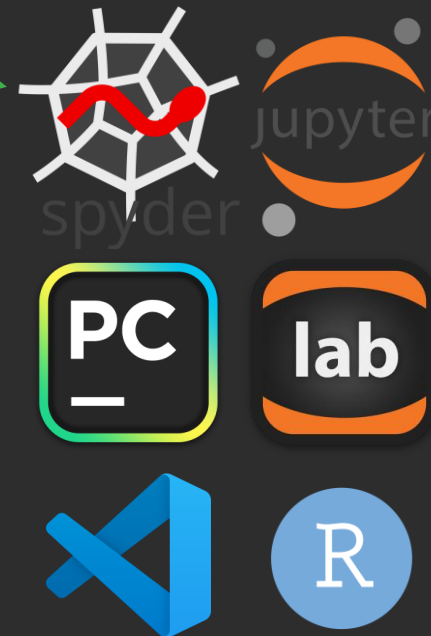
Package + Environment Manager.



Curated Package List. (base)



Editors.



ANACONDA

But this is much easier!

Developer Toolkit.

Computer.



Version Control.



Git + GitHub

Editor/IDE.



VSCode + Jupyter

Developer Seb.



Python Interpreter.



Packages Manager.



ANACONDA®

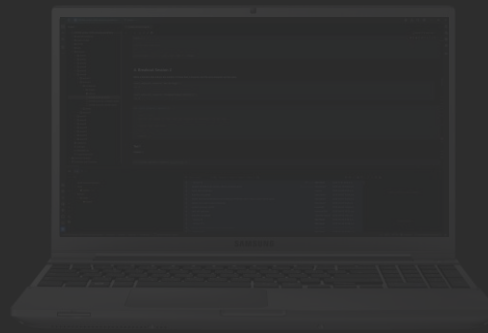
Terminal.



Anaconda Terminal

Week 11 (Don't Stress).

Computer.



Python Interpreter.



Version Control.



Git + GitHub

Packages Manager.



ANACONDA

Terminal.



Anaconda Terminal



Developer Seb.

Editor/IDE.



VSCode + Jupyter

Before Next Class.

- How Does That Work?
 - Modules -> Course Resources -> How Does That Work? (HDTW).

How Does That Work? (HDTW)

How Does That Work (HDTW) is a series of short instructional videos focused on the tools we use in this course.

[HDTW - Installing Python & Anaconda](#)

[HDTW - Anaconda Navigator](#)

[HDTW - Jupyter Notebooks](#)

[HDTW - UofT JupyterHub](#)

[HDTW - IDEs & VSCode](#)

[HDTW - Debugging \(VSCode\)](#)

[HDTW - Environments & Installing Packages](#)

- In **Tutorial 0** this week -> Set up your development environment.

The Coding Toolbox.

Week 1 | Lecture 2 (1.2)

Upcoming

- Lab 0 Released 6 pm Thursday.
- Reflection 1 Released Friday 6:00 pm.
- Tutorial sessions running all week.

if nothing else, write **#cleancode**.